

Prometheus:

How setup alerts

How alert manager sends notifications to slack, pager duty, & email

```
ubuntu@ip-172-31-1-103:~$ kubectl get svc -n monitoring
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
prometheus-alertmanager             ClusterIP    10.100.197.189 <none>         9093/TCP   10m
prometheus-alertmanager-headless    ClusterIP    None           <none>         9093/TCP   10m
prometheus-kube-state-metrics        ClusterIP    10.100.181.27  <none>         8080/TCP   10m
prometheus-prometheus-node-exporter  ClusterIP    10.100.248.40  <none>         9100/TCP   10m
prometheus-prometheus-pushgateway    ClusterIP    10.100.225.183 <none>         9091/TCP   10m
prometheus-server                    ClusterIP    10.100.69.192  <none>         80/TCP     10m
ubuntu@ip-172-31-1-103:~$ kubectl edit svc prometheus-alertmanager -n monitoring
```

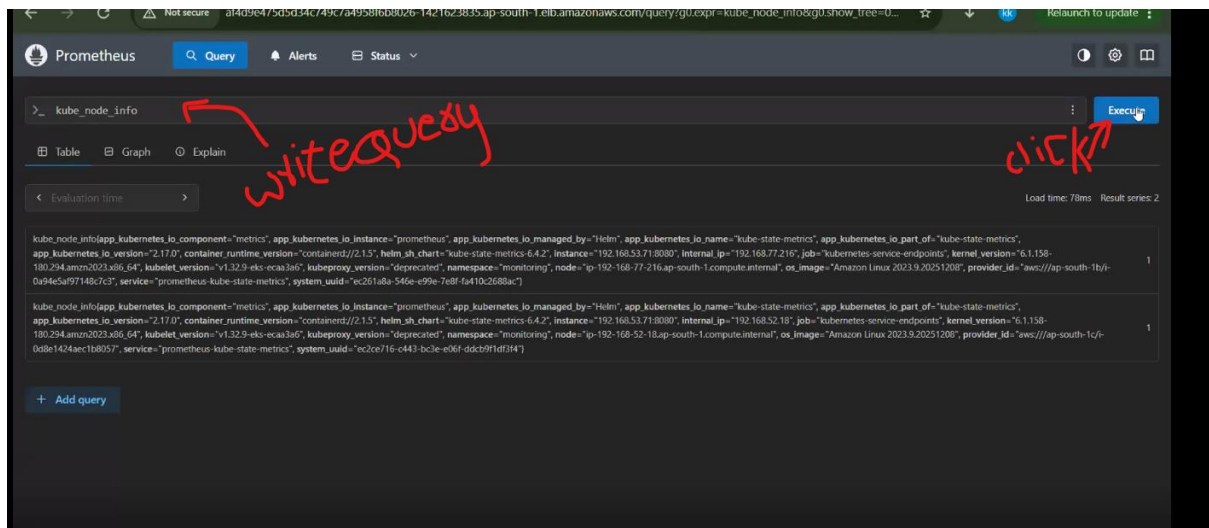
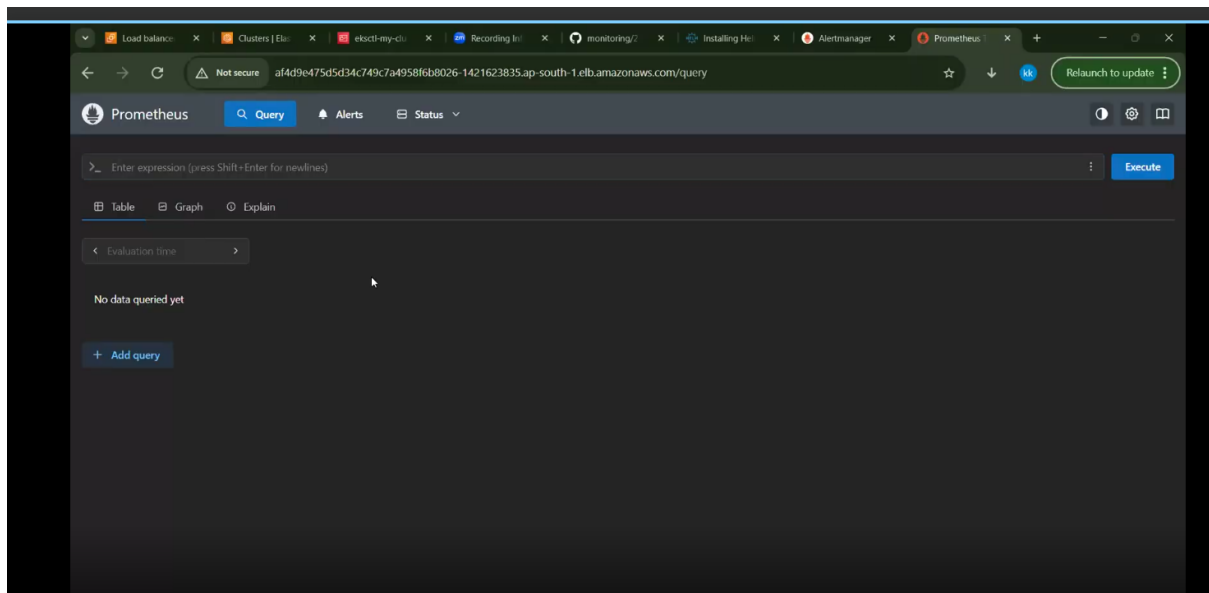
change to
LB

```
#
apiVersion: v1
kind: Service
metadata:
  annotations:
    meta.helm.sh/release-name: prometheus
    meta.helm.sh/release-namespace: monitoring
  creationTimestamp: "2025-12-14T14:08:12Z"
  labels:
    app.kubernetes.io/instance: prometheus
    app.kubernetes.io/managed-by: Helm
    app.kubernetes.io/name: alertmanager
    app.kubernetes.io/version: v0.29.0
    helm.sh/chart: alertmanager-1.29.0
  name: prometheus-alertmanager
  namespace: monitoring
  resourceVersion: "279298"
  uid: 1ea4fd3f-531a-40e7-b367-96331a2d2595
spec:
  clusterIP: 10.100.197.189
  clusterIPs:
    - 10.100.197.189
  internalTrafficPolicy: Cluster
  ipFamilies:
    - IPv4
  ipFamilyPolicy: SingleStack
  ports:
    - name: http
      port: 9093
      protocol: TCP
      targetPort: http
  selector:
    app.kubernetes.io/instance: prometheus
    app.kubernetes.io/name: alertmanager
  sessionAffinity: None
  type: LoadBalancer
```

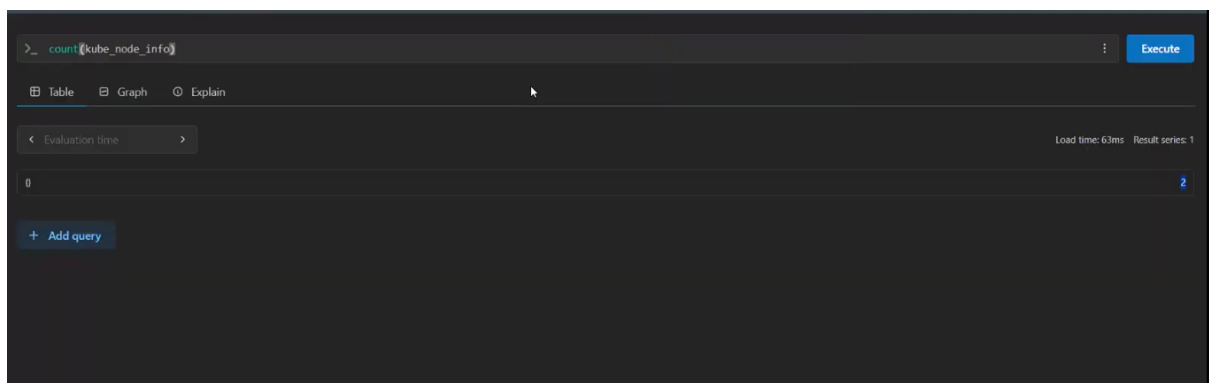
Whenever we get change to LB will get external ip---through that we can Access the application

We open Prometheus server-----dashboard

& Alert manager Dashboard-----



Above result is node info-----we are unable to understand this so-----through data source we visualize this result In Grafana dashboards.



The screenshot shows the Prometheus Query Editor interface. The query entered is `kube_pod_status_phase{phase="Running"}`. The results are displayed in a table format, showing multiple instances of the metric. A 'Timing stats' popup is visible on the right, showing the following data:

Timing stats (s):	
evalTotalTime	0.000085678
resultSortTime	0
queryPreparationTime	0.000016903
innerEvalTime	0.00006263
execQueueTime	0.000104212
execTotalTime	0.000096466
Sample stats:	
totalQueryableSamples	18
peakSamples	18

With the help of this metrics we can create dashboards in Grafana

Based on the queries we configure the alerts:

The screenshot shows a Gmail alert notification titled '2. Prometheus 14-DEC-2025 .mp4 h in Drive'. The notification details an alert for 'Firing' with the following information:

- Labels:**
 - Alertname = KubePodNotReady
 - Namespace = test-ns
 - pod = java-webapp-deploy-7559484d0-8t9t9
 - severity = warning
- Annotations:**
 - description = Pod test-ns/java-webapp-deploy-7559484d0-8t9t9 has been in a non-ready state for longer than 5 minutes.
 - summary = Pod has been in a non-ready state for more than 2 minutes.

Then login through the bastion server & check pod status

Kubectl get pod -n (ns)

Kubectl describe pod ns (test-namespcae)

Resolve the issue

100 Daily Used Prometheus (PromQL) Queries

◆ 1. Node / Host – CPU

1. CPU usage %

$100 - \text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}])) * 100$

2. CPU idle

$\text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"idle"}\}[5\text{m}]))$

3. CPU system usage

$\text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"system"}\}[5\text{m}]))$

4. CPU user usage

$\text{avg}(\text{rate}(\text{node_cpu_seconds_total}\{\text{mode}=\text{"user"}\}[5\text{m}]))$

5. CPU per node

$\text{sum by}(\text{instance})(\text{rate}(\text{node_cpu_seconds_total}[5\text{m}]))$

◆ 2. Node – Memory

6. Memory usage %

$(\text{node_memory_MemTotal_bytes} - \text{node_memory_MemAvailable_bytes})$

$\text{node_memory_MemTotal_bytes} * 100$

7. Memory used

$\text{node_memory_MemTotal_bytes} - \text{node_memory_MemAvailable_bytes}$

8. Memory available

$\text{node_memory_MemAvailable_bytes}$

9. Swap usage

$(\text{node_memory_SwapTotal_bytes} - \text{node_memory_SwapFree_bytes})$

10. Memory per node

$\text{sum by(instance)}(\text{node_memory_MemTotal_bytes})$

◆ 3. Node – Disk

11. Disk usage %

$100 * (\text{node_filesystem_size_bytes} - \text{node_filesystem_free_bytes})$

$\text{node_filesystem_size_bytes}$

12. Disk free

$\text{node_filesystem_free_bytes}$

13. Disk used

$\text{node_filesystem_size_bytes} - \text{node_filesystem_free_bytes}$

14. Disk I/O read

$\text{rate}(\text{node_disk_read_bytes_total}[5m])$

15. Disk I/O write

$\text{rate}(\text{node_disk_written_bytes_total}[5m])$

◆ 4. Node – Network

16. Network receive

$\text{rate}(\text{node_network_receive_bytes_total}[5m])$

17. Network transmit

$\text{rate}(\text{node_network_transmit_bytes_total}[5m])$

18. Network errors

$\text{rate}(\text{node_network_receive_errs_total}[5m])$

19. Dropped packets

rate(node_network_receive_drop_total[5m])

20. Network per node

sum by(instance)(rate(node_network_receive_bytes_total[5m]))

◆ 5. Kubernetes – Cluster

21. Total nodes

count(kube_node_info)

22. Nodes not ready

kube_node_status_condition{condition="Ready",status="false"}

23. Cluster CPU capacity

sum(kube_node_status_capacity_cpu_cores)

24. Cluster memory capacity

sum(kube_node_status_capacity_memory_bytes)

25. Cluster CPU usage

sum(rate(container_cpu_usage_seconds_total[5m]))

◆ 6. Kubernetes – Pods

26. Running pods

count(kube_pod_status_phase{phase="Running"})

27. Pending pods

count(kube_pod_status_phase{phase="Pending"})

28. Failed pods

count(kube_pod_status_phase{phase="Failed"})

29. Pod restarts

rate(kube_pod_container_status_restarts_total[5m])

30. Pods per namespace

count by(namespace)(kube_pod_info)

◆ 7. Kubernetes – Containers

31. Container CPU usage

rate(container_cpu_usage_seconds_total[5m])

32. Container memory usage

container_memory_working_set_bytes

33. Container memory limit

container_spec_memory_limit_bytes

34. Container OOM kills

kube_pod_container_status_terminated_reason{reason="OOMKilled"}

35. Top memory containers

topk(5, container_memory_working_set_bytes)

◆ 8. Kubernetes – Namespace

36. Namespace CPU usage

sum by(namespace)(rate(container_cpu_usage_seconds_total[5m]))

37. Namespace memory usage

sum by(namespace)(container_memory_working_set_bytes)

38. Namespace pod count

count by(namespace)(kube_pod_info)

39. Namespace restarts

sum by(namespace)(kube_pod_container_status_restarts_total)

40. Namespace limits

kube_resourcequota

◆ 9. Application Metrics

41. HTTP request rate

rate(http_requests_total[5m])

42. HTTP 5xx errors

rate(http_requests_total{status=~"5.."}[5m])

43. Error percentage

rate(http_requests_total{status=~"5.."}[5m])

/

rate(http_requests_total[5m]) * 100

44. Request latency p95

histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

45. Request latency p99

histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

◆ 10. Prometheus Health

46. Target down

up == 0

47. Scrape duration

scrape_duration_seconds

48. Scrape failures

scrape_samples_scraped == 0

49. TSDB size

prometheus_tsdb_storage_blocks_bytes

50. Active targets

count(up)

◆ 11. Alerting / Reliability

51. High CPU alert condition

avg(rate(node_cpu_seconds_total{mode!="idle"}[5m])) > 0.8

52. High memory alert

(node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes) < 0.2

53. Pod crash looping

increase(kube_pod_container_status_restarts_total[10m]) > 3

54. Node down

up{job="node"} == 0

55. Disk full

node_filesystem_free_bytes / node_filesystem_size_bytes < 0.1

◆ 12. Advanced / SRE

56. Burn rate (SLI)

(rate(http_requests_total{status=~"5.."}[5m]))

57. QPS

```
sum(rate(http_requests_total[1m]))
```

58. Top CPU pods

```
topk(5, rate(container_cpu_usage_seconds_total[5m]))
```

59. Top memory namespaces

```
topk(5, sum by(namespace)(container_memory_working_set_bytes))
```

60. Node pressure

```
node_pressure_memory_waiting_seconds_total
```

◆ 13. Misc Daily Queries

61. Load average

```
node_load1
```

62. File descriptors

```
node_filefd_allocated
```

63. Time drift

```
node_timex_offset_seconds
```

64. Kubernetes API latency

```
apiserver_request_duration_seconds_bucket
```

65. API errors

```
apiserver_request_total{code=~"5.."}  
  


---


```

◆ 14. Capacity Planning

66. CPU requests vs usage

```
sum(container_spec_cpu_quota) / sum(rate(container_cpu_usage_seconds_total[5m]))
```

67. Memory requests vs usage

```
sum(container_spec_memory_limit_bytes) /  
sum(container_memory_working_set_bytes)
```

68. Node allocatable CPU

```
kube_node_status_allocatable_cpu_cores
```

69. Node allocatable memory

```
kube_node_status_allocatable_memory_bytes
```

70. Pod density per node

count by(node)(kube_pod_info)

◆ 15. Final Set (Common Interview Picks)

71. Running vs failed pods

kube_pod_status_phase

72. Restart rate

rate(kube_pod_container_status_restarts_total[5m])

73. Pod CPU throttling

rate(container_cpu_cfs_throttled_seconds_total[5m])

74. Node readiness

kube_node_status_condition{condition="Ready"}

75. Control plane health

up{job="kube-apiserver"}

◆ Bonus (76–100): Variants you'll *actually* tweak daily

- Same queries scoped by:
 - instance
 - namespace
 - pod
 - container
 - job
 - cluster

These 25 come from filtering + grouping the above.

🌀 Interview Closing Line

"In daily operations, we mainly use PromQL for node health, Kubernetes pod status, resource utilization, application SLIs, and alert conditions. Most dashboards are composed by grouping, filtering, and aggregating these core metrics."